COMP3230 Principles of Operating Systems

Programming Assignment One

DUE DATE	TOTAL	VERSION
Oct. 16, 2026, at 10:29 am	15 points	1.0

Programming Exercise – Yet Another Shell (yash) Program

Objectives

1. An assessment task related to ILO4 [Practicability] – "demonstrate knowledge of applying system software and tools available in a modern operating system for software development".
2. A learning activity related to ILO 2a.
3. The goals of this programming assignment are:
    - to have hands-on practice in designing and developing a shell program, which involves the creation, management, and coordination of multiple processes;
    - to learn how to use various important Unix system functions:
        - to perform process creation and program execution;
        - to support interaction between processes by using signals and pipes; and
        - to obtain the process's runtime statistics.

Task

A shell program, commonly known as a command interpreter, acts as the user interface to the operating system and allows it to interpret your commands. For example, when you enter the **`ls -la`** command, the shell executes the system utility **`ls`** for you.

A shell program can be used interactively or for batch processing.

You are going to implement a C program that acts as an ***interactive shell program*** named ***3230yash***. This program supports the following features (full details are provided in the Specification section):

1. The program (3230yash) accepts a command from the user and executes the corresponding program with the given argument list.
2. It can locate and execute any valid program (i.e., compiled programs) by specifying an absolute path (starting with /), a relative path (starting with ./), or by searching the directories listed in the $PATH environment variable.
3. It has two built-in commands:
    - **exit** command that terminates the 3230yash program; once the program starts, it continuously accepts commands from the user until receiving the exit command.
    - **watch** command that monitors the resource usage of a running child process until it terminates.
4. It allows the use of | (pipe) operators to chain commands together.
5. The 3230yash process should not be terminated by Ctrl-C (SIGINT signal).

Specification

The behavior of the 3230yash program is specified below.

- Like a traditional shell program, when the 3230yash process is ready to accept input, it displays a prompt and waits for input from the user:

```
## 3230yash >>
```

After accepting a line of input from the user, it parses the input line to construct the command name(s) and associated argument list(s), then creates child process(es) to execute the command(s). We can assume that the command line is limited to a maximum of 1024 characters and contains up to 30 strings, including the "|" symbol.

After creating a child process, the 3230yash process should wait for it to terminate before displaying the prompt for the next user input.

- The 3230yash process should be able to locate and execute any program that can be found using:
  - the absolute path specified in the command line, e.g.

```
## 3230yash >>    /home/tmchan/a.out
```

  - the relative path specified in the command line, e.g.

```
## 3230yash >>    ./a.out
```

  - a search of the directories listed in the $PATH environment variable, e.g.

```
## 3230yash >>    gcc hello.c
```

where *gcc* is in the directory `/usr/bin`, and
`$PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin`

- If the program cannot be located or executed, 3230yash displays an error message:

```
## 3230yash >>    simp
3230yash: 'simp': No such file or directory
## 3230yash >>    /bin/simp
3230yash: '/bin/simp': No such file or directory
## 3230yash >>    ./start.sh
3230yash: './start.sh': Permission denied
```

- | symbol – If | (pipe) symbols appear between commands in an input line, 3230yash creates multiple child processes, connects the standard output of the command before each pipe, and links it to the standard input of the command after the pipe. That is, each command except the first reads the previous command's output. We can assume that the user will not enter more than 4 pipes (i.e., 5 commands) in an input command line. For example:

```
## 3230yash >>    cat TP-2627.html | grep table | wc -l
6
## 3230yash >>
```

We can assume that it is invalid to place the | symbol as the first or last character on the input line. Two consecutive | symbols (e.g., '||' or '| |') without a command between them are also invalid.

```
## 3230yash >> | wc -l
3230yash: Incorrect pipe sequence
## 3230yash >> cat abc.txt |
3230yash: Incorrect pipe sequence
## 3230yash >> cat hello.c | | wc -l
3230yash: should not have two consecutive | without in-between command
## 3230yash >>
```

Please refer to Tutorial 2 to learn how to use *`pipe()`* and *`dup()`* system functions to set up a pipe between two processes.

- **Built-in command: exit** – If the user enters the exit command, 3230yash should release all its resources and then terminate, after which the standard shell prompt reappears. For example:

```
## 3230yash >> exit
3230yash: Terminated
root@ff860ab794d9:/home/tmchan/c3230# ps
  PID TTY      STAT   TIME COMMAND
    1 pts/0    Ss     0:00 bash
   79 pts/0    R+     0:00 ps
root@ff860ab794d9:/home/tmchan/c3230#
```

If the *`exit`* command has other arguments, 3230yash should not treat it as a valid exit request and should not terminate. In addition, if *`exit`* is not the first word in the command line, 3230yash should not treat it as the exit command.

```
## 3230yash >>    not exit
3230yash: 'not': No such file or directory
## 3230yash >>    exit now
3230yash: "exit" with other arguments!!!
## 3230yash >>
```

- **Built-in command: watch** – The watch command periodically monitors the resource usage of a running child process and reports the data when the process terminates. Because the watch command can manage only one command at a time, it cannot monitor a sequence of piped commands.

```
## 3230yash >> watch ps f
  PID TTY      STAT   TIME COMMAND
   20 pts/2    Ss     0:00 /bin/sh
   26 pts/2    S+     0:00  \_ bash
    9 pts/1    Ss     0:00 /bin/sh
   15 pts/1    S      0:00  \_ bash
   19 pts/1    S+     0:00      \_ ./yash
   48 pts/1    R+     0:00          \_ ps f
    1 pts/0    Ss+    0:00 bash

STATE CPUID UTIME STIME VSIZE     MINFLT MAJFLT
R     24    0.00  0.00  2748416   10     0
Z     24    0.00  0.00  0         370    0

## 3230yash >> watch ./add

STATE CPUID UTIME STIME VSIZE     MINFLT MAJFLT
R     18    0.00  0.00  2748416   10     0
R     18    0.49  0.00  2609152   61     2
R     18    0.99  0.00  2609152   61     2
Z     18    1.27  0.00  0         65     2
## 3230yash >> watch cat hello.c | wc -l
3230yash: Cannot watch a pipe sequence
## 3230yash >>
```

All process status information managed by the Linux kernel can be found in the `/proc/{pid}/stat` file. This assignment requires you to use the `/proc` filesystem to extract a process's runtime statistics. Please refer to Tutorial 2 to learn how to retrieve these statistics from the `/proc` filesystem. A more detailed description of the `/proc/{pid}/stat` file is available at https://man7.org/linux/man-pages/man5/proc_pid_stat.5.html

Here is a summary of the information in the output.

STATE	Indicates the process's current state: Running, Sleeping, Zombie, ...
CPUID	The CPU number on which the process last executed
UTIME	The time (in seconds) that the process spent in user mode
STIME	The time (in seconds) that the process spent in kernel mode
VSIZE	The process's virtual memory usage in bytes
MINFLT	The number of minor faults that did not require loading a memory page from disk
MAJFLT	The number of major faults that required loading a memory page from disk

The 3230yash process uses the following mechanism to monitor the target child process.

  - Before the child process begins executing the target command, 3230yash captures an initial snapshot of the child's resource usage. The child process then starts executing the command.
  - Every 500 milliseconds, 3230yash takes a new snapshot of the child process's resource usage and continues doing so until the child process terminates.
  - Once the child process has finished and produced its output, 3230yash displays all collected resource usage information for the terminated child process.

You will need to use the non-blocking mode of the *`waitid()`* system call.

- The 3230yash process is required to handle the SIGINT signal. The corresponding signal handlers should be implemented.

– **SIGINT signal:** The 3230yash process and its child processes are required to respond to the SIGINT signal (generated by pressing Ctrl-C or using the kill command) according to the following guideline:

The child process should respond to the SIGINT signal according to the program's predefined behavior; therefore, some child processes may terminate, while others may not.

```
## 3230yash >>    ./forever
^CReceives SIGINT!! IGNORE IT :)
^CReceives SIGINT!! IGNORE IT :)
^CReceives SIGINT!! IGNORE IT :)
^CReceives SIGINT!! IGNORE IT :)
forever:    Killed
## 3230yash >>

## 3230yash >>    ./loopf 10
^Cloopf:    Interrupt
## 3230yash >>
```

If a process is terminated by a signal, 3230yash should display the program's name together with the type of signal that caused it to terminate. For example, if it detects SIGINT, it prints "Interrupt"; if it detects SIGKILL, it prints "Killed". Please refer to Mini-lab 2 and Tutorial 2 to learn how to handle signals and display appropriate messages.

The 3230yash process should not be terminated by SIGINT. When the user presses Ctrl-C while the 3230yash process is waiting for input, 3230yash should display a new prompt:

```
## 3230yash >>  ^C
## 3230yash >>  ^C
## 3230yash >>  ^C
## 3230yash >>
```

Documentation

1. At the beginning of the submitted source code, clearly state the following:
   - Student name and number
   - Development platform
   - Remarks describing how much you have completed (see Grading Criteria)
2. Use detailed inline comments so that others can easily understand your code.

Computer Platform to Use

For this assignment, you can develop and test your program on any Linux platform, but you must ensure that it runs correctly on the **workbench2** Linux server, which the tutors will use for grading. Your program must be written in C and compile successfully with `gcc` on the server. We recommend developing and testing your program on your own machine (WSL2 or an Ubuntu Docker container). After testing it thoroughly locally, upload the program to **workbench2** for the final tests.

Note that the only server for COMP3230 is workbench2.cs.hku.hk. Please do not use any other CS department server, especially academy11 and academy21, because they are reserved for other courses. As the department and university have tightened security controls, all users must first connect to the HKU network via HKUVPN before they can access the CS department's servers. If you cannot log in to workbench2, please contact the tutors for help.

Submission

Submit your program to the Programming Assignment 1 submission page on the course Moodle website. Name your program using this format: **`3230yash_{UID}.c`** (replace `{UID}` with your HKU student number).

As Moodle may not accept source-code submissions, compress your program in zip or tgz format before uploading.

Grading Criteria

1. Your submission will be primarily tested on workbench2. Make sure that your program can be compiled without errors. Otherwise, we cannot test your submission, and you will receive a mark of zero.
2. Because the tutors will check your source code, please write readable code (i.e., use consistent coding conventions and sufficient comments) so that you do not lose marks because of ambiguity.

Documentation (–1 point if not provided)	• Include the comments needed to explain the program logic clearly • Include the required student information at the beginning of the program

Category	Component	Criteria
Correctness of the program 15 points	**Process creation and execution 4.5 points**	• Should be able to print "## 3230yash >> " and accept user input • Should be able to use a new process to execute the input command • Can locate and execute a command using a full path, a relative path, or the standard PATH • Can execute a command with any number of arguments • Can handle error situations correctly, e.g., an incorrect filename or path, or a non-binary file • Should wait for the child process to complete before accepting the next request • Should report the signal that terminated the child process • Should allow the user to execute multiple commands (with arguments) one after another • Should remove all zombie processes
	Process creation and execution – use of '|' 4 points	• Correct use of the | symbol; can report improper use of pipes • Can execute two commands connected by a pipe • Can execute two commands, each with any number of arguments, connected by a pipe • Can execute up to 5 commands, each with any number of arguments, connected by a sequence of pipes
	SIGINT signals 2 points	• The 3230yash process should respond correctly to the SIGINT signal • Child processes may be terminated by the SIGINT signal
	Built-in command watch 4 points	• Correct use of the watch command; can report improper usage • Should be able to report process information before executing the user's command • Should report process information periodically during execution • Should report process information after the process finishes • Should display the information after the child process has terminated
	Built-in command exit 0.5 points	• Correct use of the exit command; can report improper usage

Plagiarism

Plagiarism is a very serious offense. Students should understand what constitutes plagiarism, the consequences of committing plagiarism, and how to avoid it. Please note that we may ask you to explain how your program works, and we may use software tools to detect software plagiarism.

Using GenAI

You are allowed to use Generative AI to help you complete the assignment. If you use GenAI, you **SHOULD** include a report clearly describing how you used GenAI, including:

- The GenAI models you used.
- The prompts or questions you submitted to the GenAI model and its responses. If you adopt any generated code snippets, you SHOULD clearly identify them.

If we suspect plagiarism in your submission, it is not acceptable to justify it by stating that you used GenAI for coding guidance and that its output merely mirrors that of other students. While GenAI can be a valuable tool for exploring different techniques, you often need to adjust generated code snippets to tailor them to the specific task.